

ДИПЛОМЫН УРЬДЧИЛСАН ХАМГААЛАЛТ · 2026

Linksy

Сайн байна уу. Миний дипломын ажил **Linksy** нэртэй нийгмийн сүлжээ платформ. Энэ нь зүгээр нэг feed, like, comment-тай энгийн апп биш юм. Доорх боломжуудыг бүгдийг нь нэг систем дотор бодитоор хэрэгжүүлсэн: *хувийн тойргийн мессеж, шифрлэгдсэн чат, дуу/видео дуудлага, гар утас + browser-руу push мэдэгдэл, creator-уудын XP оноо систем.*

NEXT.JS 16

REACT 19

TYPESCRIPT

POSTGRESQL 16

PRISMA 7

REDIS

DOCKER

WEBRTC

E2EE

SENTRY

Linksy юуг шийдэх вэ, ямар боломжуудтай вэ?

Эхлээд асуудлаас эхэлье. Facebook, Instagram гэх мэт нийгмийн сүлжээнүүд хэрэглэгчийн өгөгдлийг хязгааргүй цуглуулдаг, мөн ямар пост хэнд харагдахыг "нууц алгоритм" шийддэг. Linksy үүний оронд хэрэглэгч өөрөө **хэн пост-ыг нь үзэх**-ийг (Public, найз, эсвэл "Close circle" гэсэн ойрын тойрог) шийдэх боломжтой. Эрэмбэлэлт нь нууц биш — **like + comment-ийн тоо × шинэлэг байдал** гэсэн ил тод томъёотой.

Системийн үндсэн 10 боломжийг танилцуулъя:

Creator Mode + XP оноо

2 пост эсвэл 10 like/comment хүлээн авсан хэрэглэгч идэвхжүүлж болно. Пост бичих=10, like ирэх=2, comment=3, follow=4, өдөр бүр нэвтрэх=5 оноо. Subscription tier-ээс хамаарч оноо үржинэ. Redis-д leaderboard.

Creator Dashboard

Үзэлт, like, comment, save, repost, follow-ийн жинхэнэ статистик. Өмнөх 7/30/90 хоногтой харьцуулсан өсөлт хувиар. Өдрийн график, шилдэг 5 пост.

Authentication

Email + нууц үгээр бүртгэх/нэвтрэх. Token нь 15 минут тутамд солигдох (security), 2FA код, email баталгаажуулалт, нууц үг сэргээх, олон удаа буруу оруулбал түр түгжих.

5-н горимтой Feed

For You

(бүгд),

Friends

(зөвхөн дагасан),

Close circle

(ойрын тойрог),

Creator

(creator-ууд),

Discover

(алгоритмаар сонгогдсон шинэ хүмүүс).

Пост & Story

1-4 зураг/видео, санал асуулга, цаг товлох, draft, цомог, comment модераци, хамтран зохиогчид, hashtag/mention, харааны бэрхшээлд зориулсан alt бичиг заавал.

Бодит цагийн чат

1-1 эсвэл group чат, live шинэчлэлт, "бичиж байна..." indicator, унших баталгаа, дуут мессеж, тогтоосон хугацаа өнгөрсний дараа арилах мессеж.

End-to-End шифрлэлт

Мессеж зөвхөн илгээгч/хүлээн авагч хоёрт уншиж болохуйц. Сервер plaintext харахгүй — зөвхөн шифрлэгдсэн ciphertext хадгалдаг. Хувийн түлхүүр browser-аас гадагш гарахгүй.

Дуу/Видео дуудлага

WebRTC — медиа stream нь peer-to-peer (хэрэглэгч хооронд шууд), сервер дамждаггүй. NAT-ийн ард байгаа хэрэглэгчдэд coturn TURN сервер ашигладаг.

Гурван суваг Push

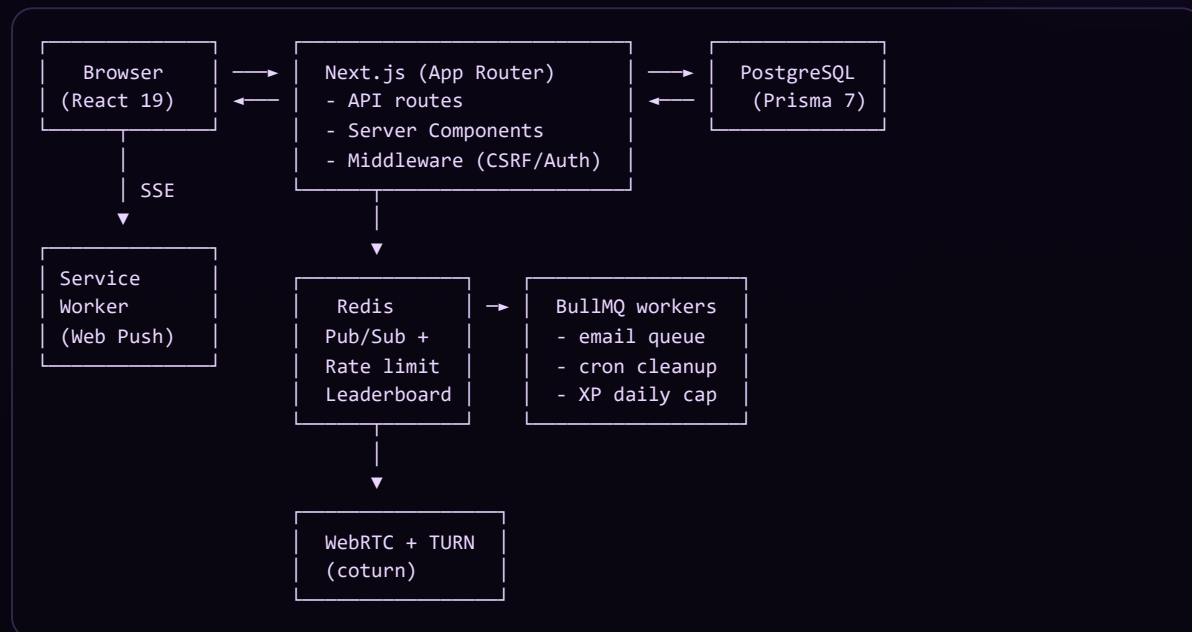
Browser, Android (FCM), iOS (APNs) — гурван өөр платформд нэг систем мэдэгдэл илгээдэг. Хэрэглэгчийн "чимээгүй цагийн" тохиргоог хүндэтгэнэ.

6 хэл + GDPR + Хяналт

Англи, Монгол, Хятад, Япон, Солонгос, Араб (баруунаас зүүн). Cookie consent banner, өгөгдлөө татаж авах эрх. Алдааг Sentry-р барих, лог-ыг pino-р цуглуулах.

Системийн зураглал

Дараагийн алхамд систем хэрхэн зохион байгуулагдсаныг тайлбарлая. **Next.js App Router** нь сервер болон клиент хоёрыг нэг кодын баазад нэгтгэн ажиллуулдаг тул хоёр серверийг тусдаа deploy хийх шаардлагагүй. Бодит цагийн боломжуудад **Server-Sent Events** ба **Redis Pub/Sub** ашиглаж, олон Node.js instance дээр event-ийг fan-out хийнэ.



ШИФРЛЭГДСЭН МЕССЕЖ 4 АЛХМААР ЯВАГДДАГ

1

Хувийн түлхүүр үүсгэх

Хэрэглэгч browser-ээрээ нийтийн (public) болон хувийн (private) хоёр түлхүүр үүсгэдэг. Зөвхөн нийтийн түлхүүрийг л серверт илгээнэ. **Хувийн түлхүүр browser дотроос гадагш гарахгүй.**

2

Хоёр хүний хооронд нууц утга тохирох

Илгээгч нөгөө хүний нийтийн түлхүүрийг авна. Дараа нь хоёр хүний түлхүүрүүдийг хольж зөвхөн тэр хоёрт мэдэгдэх **нэг ижил нууц утга**-ыг гаргаж авдаг. Энэ нь сервер хэзээ ч мэдэхгүй утга.

3

Мессежийг шифрлээд илгээх

Илгээгч мессежээ тэр нууц утгаар шифрлээд серверт илгээнэ. Сервер задлахгүй — гарын үсэг шиг харагдах ciphertext-ийг л харна. Хэрэв client алдаагаар plaintext илгээвэл сервер шууд "" болгоно.

4

Хүлээн авагчид хүргэх ба задлах

Сервер шифрлэгдсэн мессежийг хүлээн авагчид яаралтай хүргэнэ (SSE-ээр). Хүлээн авагч өөрийн хувийн түлхүүрээрээ задлаж жинхэнэ мессежийг уншина.

Tech stack ба сонголтын үндэслэл

Энэ хэсэгт ашигласан технологи бүрийг яагаад сонгосноо тайлбарлая. Технологийн нэрийг жагсаах нь хангалтгүй тул сонголтын логигоо тэр чигээр нь танилцуулна:

Next.js 16

FRONTEND +
BACKEND

Яагаад: Хуудас (frontend) болон API хоёрыг нэг кодын дотор бичиж нэг газар deploy хийнэ. Хоёр өөр сервер удирдах шаардлагагүй. Бүх API дээр auth, CSRF, rate-limit зэргийг middleware нэг газар хэрэгжүүлнэ.

React 19 + TypeScript UI БҮТЭЦ

Яагаад: React нь UI-г жижиг, давтан хэрэглэх боломжтой component-ууд болгож хуваадаг. TypeScript нь алдааг код хийж байх үед барьдаг учраас том төсөлд refactoring аюулгүй болно.

PostgreSQL 16 ҮНДСЭН ӨГӨГДЛИЙН САН

Яагаад: Хэрэглэгч ↔ пост ↔ комент гэх мэт олон харилцан холбоотой өгөгдөлд таарна. Олон үйлдлийг нэг "atom" болгож хэрэв нэг нь алдвал бүгдийг буцаах боломжтой. Аналитикт зориулсан хүчирхэг SQL функцуудтай.

Prisma 7 ORM (DB ↔ КОД ГҮҮР)

Яагаад: Schema файлаас TypeScript type-уудыг автоматаар үүсгэдэг. Хэрэв DB schema өөрчлөгдвөл код шууд алдаа заана. Migration-уудыг хянахад CI дээр автомат шалгалт хийнэ.

Redis ХУРДАН САНАХ ОЙ

Яагаад: Гурван өөр ажилд нэг систем: (1) олон удаа нэвтрэх оролдлогыг тоолох, (2) олон сервер хооронд real-time event дамжуулах, (3) Creator XP-ийн leaderboard. Бүгд миллисекундын хурдтай.

BullMQ BACKGROUND АЖЛУУД

Яагаад: Email илгээх, цэвэрлэгээний cron гэх мэт удаан ажлуудыг HTTP хүсэлтээс салгаж тусдаа процессоор гүйцэтгэдэг. Алдаатай ажлыг автоматаар дахин оролддог.

WebRTC + coturn ДУУ/ВИДЕО ДУУДЛАГА

Яагаад: WebRTC нь медиа stream-ийг хэрэглэгч хоёр хооронд **шууд** дамжуулдаг тул сервер дамждаггүй — bandwidth идэхгүй. NAT-ийн ард байгаа хэрэглэгчдэд coturn TURN сервер нь "гүүр" хийж өгдөг.

Web Crypto API ШИФРЛЭЛТИЙН АЛХАМ

Яагаад: Browser-т өөрөө шингэсэн API учраас тусдаа library суулгах шаардлагагүй, build хэмжээг өсгөхгүй. Шифрлэлтийн алгоритмуудыг компьютерийн hardware-аар хурдан гүйцэтгэдэг.

Бусад чухал сонголтууд

Үргэлжлүүлэн validation, real-time push, observability, deployment талын сонголтуудаа тайлбарлая:

Zod

ОРОЛТЫН
ШАЛГАЛТ

Яагаад: Хэрэглэгчийн API руу илгээх өгөгдлийг бүгдийг нэг газар (schema-аар) шалгадаг. Frontend form ба backend нэг ижил schema reuse хийдэг тул хоёр газар адил дүрэм бичих шаардлагагүй.

Zustand

CLIENT STATE

Яагаад: Redux-ээс хамаагүй хөнгөн (1KB), бараг код бичихгүйгээр state удирдана. Жижиг store-уудад (theme, notification) хангалттай.

VAPID · FCM ·

APNs

PUSH МЭДЭГДЭЛ

Яагаад: Гурван өөр платформ — browser (VAPID), Android (FCM), iOS (APNs) — тус бүрд тусгай суваг ашигладаг. Бид гурвыг нэг dispatcher-аар нэгтгэснээр feature код суваг бүрийн дэлгэрэнгүйг мэдэх шаардлагагүй.

Server-Sent

Events

СЕРВЕР → CLIENT
LIVE

Яагаад: WebSocket-ийн оронд SSE-ийг сонгосон. SSE нь HTTP дээр явдаг тул firewall-аар амар, автомат дахин холбогддог. Бид зөвхөн сервер→client чиглэлд мессеж явуулна — хоёр чиглэлтэй WebSocket хэрэггүй.

DOMPurify

XSS ХАЛДЛАГА
ХАМГААЛАЛТ

Яагаад: Хэрэглэгчийн илгээсэн HTML (rich-text comment гэх мэт) дотор аюултай script байж болзошгүй. DOMPurify нь HTML-ийг "цэвэрлэж" зөвхөн зөвшөөрсөн tag-уудыг үлдээдэг.

Sharp + ffmpeg

ЗУРАГ/ВИДЕО
БОЛОВСРУУЛАЛТ

Яагаад: Зургийг WebP формат руу шилжүүлэн хэмжээг 60-70% багасгана. Видеоны thumbnail (хавтасны зураг) автомат үүсгэнэ. Хоёул native binary учраас Node-ийн main thread-ийг саатуулахгүй.

Sentry + pino

АЛДАА & ЛОГ

Яагаад: Sentry — production дээр гарсан алдааг автоматаар alert-той цуглуулна. Pino — JSON формат лог, дэлхийн хамгийн хурдан Node logger-уудын нэг — server-т ачаалал бараг өгөхгүй.

Docker +

Compose

ОРЧНЫ
НЭГДМЭЛЖИЛТ

Яагаад: "Энэ миний компьютер дээр ажиллана" гэдэг асуудлыг шийднэ. docker compose up гэсэн нэг команд нь app, PostgreSQL, Redis, coturn зэрэг бүх системийг нэгэн зэрэг ажиллуулна.

Defense-in-depth ба real-time архитектур

Аюулгүй байдалд би **defense-in-depth** буюу олон давхаргат хамгаалалтын зарчмыг хэрэгжүүлсэн. Нэг хамгаалалт нэвтэрсэн ч дараагийнх нь ажиллана. Доорх хүснэгтэд гол давхаргуудыг нэгтгэсэн:

12 ДАВХАРГАТ ХАМГААЛАЛТ

ДАВХАРГА	ЮУНААС СЭРГИЙЛДЭГ ВЭ?
Нэвтрэлт	Token-ийг 15 минут тутамд солих, refresh token-ийг hash хийгээд DB-д хадгалах, 2FA код
Brute-force халдлага	1 минутад 5+ удаа буруу нууц үг оруулбал нэр-эзний нэр болон IP-г 15 минут түр түгжих
Сул нууц үг	Том+жижиг үсэг + тоо + тэмдэгт заавал, 1000 гаруй нийтлэг нууц үгийг хориглох
CSRF халдлага	Cookie + header-т ижил token шаардах — өөр сайтаас хэрэглэгчийн өмнөөс үйлдэл хийхээс сэргийлнэ
Security headers	HSTS (HTTPS заавал), X-Frame DENY (iframe-д суулгахаас сэргийлнэ), CSP (зөвхөн зөвшөөрсөн source-аас script ачаалах)
Гадны API хандалт	CORS allow-list: зөвшөөрсөн origin-аас л хүсэлт хүлээн авна
SQL injection	Бүх raw SQL нь параметр-аар явдаг, хэрэглэгчийн оролтыг шууд SQL-д хольдоггүй
XSS халдлага	Хэрэглэгчийн илгээсэн HTML-ийг DOMPurify-ээр цэвэрлэж байж л render-дэг
End-to-end шифрлэлт	Мессеж сервер дээр шифрлэгдэж хадгалагдана. Серверийн админ ч хэзээ ч уншиж чадахгүй
Webhook хуурамчлал	Stripe, push webhook-ийн HMAC гарын үсгийг шалгана
Аудит лог	Login, password солих, 2FA, дуудлагын төлөв — бүгдийг бүртгэдэг
GDPR нийцэл	Cookie зөвшөөрөл асуух, хэрэглэгч өөрийн өгөгдлийг татаж авах эрх

REAL-TIME BACKBONE

Бодит цагийн харилцаа нь дөрвөн үндсэн системээс бүрдэнэ:

SSE + Redis Pub/Sub

Бүх live feature нэг архитектур дээр: messages, typing, notification, story view, call signaling. Олон Node instance дээр Redis-аар fan-out.

WebRTC + coturn

Signaling: SDP+ICE Redis pub/sub, 240/min rate-limit. Media: P2P only. NAT traversal: ephemeral HMAC-SHA1 credential per user.

Disappearing messages

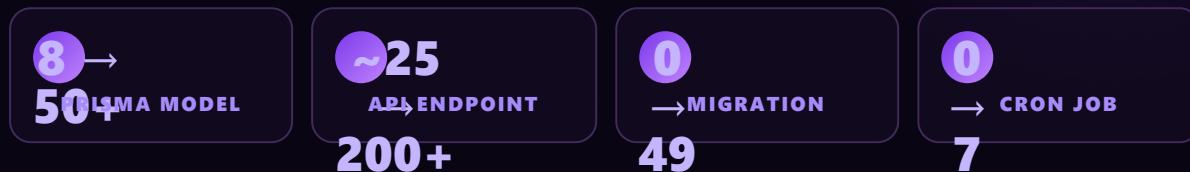
OFF · TIMED · AFTER_READ mode, 60..604800 сек TTL. Snapshot нь Message row-д хадгалагдах тул policy дараа өөрчлөгдсөн ч хуучин мессеж тэр чигтээ үлдэнэ.

Multi-channel push

Нэг dispatcher хэрэглэгчийн бүх PushSubscription-уудад fan-out. Quiet hours per-TZ minute-of-day. Unsubscribed token автоматаар цэвэрлэгдэнэ.

Аюулгүй байдал, нэвтрэлт, өгөгдлийн загвар

April сардын танилцуулга үед апп нь зөвхөн **JWT cookie auth**, **8 ширхэг Prisma model**, үндсэн like/comment/follow/notification урсгал л байсан. Тэр цагаас хойш өнөөдрийг хүртэл production-түвшний дараах **системүүд бодитоор хэрэгжсэн**. Эхний хэсэгт — нэвтрэлт, аюулгүй байдал, өгөгдлийн бүтэц:



🔒 НЭВТРЭЛТИЙН СИСТЕМ (ШИНЭЭР БҮХЭЛД НЬ)

2FA (TOTP) + backup codes

Google Authenticator-той бэлэн. QR код setup, 6 оронтой код, 10 ширхэг нэг удаагийн backup code, Settings → Security-аас идэвхжүүлэх/унтраах.

Email verification + Password reset

Token-based verify (24ц TTL), нууц үг сэргээх (60 мин TTL), Resend/SendGrid + React-email template (welcome/verify/reset/login-alert/digest).

Refresh token rotation

15-мин access JWT + opaque refresh (SHA-256 hash) 30-day sliding window. Ашигласан бүрт шинэ refresh issuance. Logout → revoke.

Brute-force logout

Redis rate-limit (IP 40/15мин + identifier 5/1мин), 5 буруу нууц → 15 мин account logout ('User.failedLoginAttempts/lockedUntil'), 429 + Retry-After.

Password policy + Common blocklist

Том+жижиг+тоо+тэмдэгт заавал, 8-128 урт, дахигдах тэмдэгт хориглох, 1000+ "password123", он+суурь хэв маяг бүхий Set blocklist.

Session list + revoke

Идэвхтэй device-уудын жагсаалт (IP + UA + ipgeo + lastSeen) + remote revoke. "Sign out other sessions" Settings-д.

🛡️ WEB-АЮУЛГҮЙ БАЙДАЛ (12-ДАВХАРГАТ "DEFENSE-IN-DEPTH")

ДАВХАРГА	ХЭРЭГЖИЛТ
Security headers	CSP (script/style/img/connect/frame source-уудыг нягт), HSTS (production-д max-age 2 жил, preload), X-Frame DENY, X-Content-Type nosniff, Referrer strict-origin, Permissions-Policy (cam/mic = self)
CSRF double-submit	`linksy_csrf` cookie (JS-readable) + `x-csrf-token` header нэг утгатай байх; mutating method-д middleware шалгана; webhook + auth flow чөлөөлсөн
CORS allow-list	`CORS_ALLOWED_ORIGINS` env-p whitelist; OPTIONS preflight 204 буцаах middleware; зөвхөн зөвшөөрөгдсөн Origin-д Access-Control-Allow-Credentials
SQL injection audit	Бүх `\$queryRaw`/`\${executeRaw}` дуудлагыг шалгасан ('Prisma.sql' + параметр); `\$queryRawUnsafe` ашиглалт 0; шинэ raw SQL-д нь rule
Webhook signature	`STRIPE_WEBHOOK_SECRET` (Stripe HMAC + timestamp tolerance) + `PUSH_WEBHOOK_HMAC_SECRET` (X-Hub-Signature-256); failed → 400

Audit log +
retention

`AuditLog` model + 6yx security event; `/api/cron/audit-log-retention` (default 365д), батч DELETE
5000/run

Real-time, мессеж, дуу/видео, push, медиа

Хоёрдугаар хэсэг — танилцуулгад "WebSocket-аар real-time chat болгоно" гэж *ирээдүйн боломж*-оор жагсаасан зүйлс бүгд **бодитоор хэрэгжсэн**: SSE-based real-time backbone, end-to-end шифрлэлт, WebRTC дуу/видео, олон сувгийн push мэдэгдэл, бүрэн медиа pipeline.

REAL-TIME BACKBONE (SSE + REDIS PUB/SUB)

5 ширхэг live stream

``/messages/stream`` (шинэ мессеж), ``/typing/stream`` (бичиж байна), ``/notifications/stream`` (like/comment/follow), ``/conversations/stream`` (DM list), ``/presence/stream`` (online dot).

Olon Node-instance fan-out

``notification-bus.ts``, ``conversation-message-bus.ts``, ``user-inbox-bus.ts``, ``presence-bus.ts``, ``typing-bus.ts``, ``call-signal-bus.ts`` — Redis Pub/Sub давхагра.

Online presence

Heartbeat 30s + Redis SETEX 90s TTL, sendBeacon offline-д; chat header дээр green dot + "Active now" / "Last seen Xm ago".

Read receipts шууд

``markRead`` дараа ``convmsg:read`` event publish, sender UI-д "Read" tick instant харагдана (өмнөх 30s polling → ~50ms SSE push).

END-TO-END ENCRYPTION + WEBRTC + DISAPPEARING

E2EE (ECDH + AES-GCM)

Web Crypto P-256 ECDH + X3DH-style key agreement + AES-256-GCM. Private key browser IndexedDB-д л; сервер plaintext татгалзана, ``E2EEIdentity`` + ``OneTimePreKey`` (atomic OPK consume).

WebRTC дуу/видео

1:1, SDP/ICE Redis pub/sub signaling, coturn HMAC-SHA1 ephemeral creds, missed-call cleanup cron (RINGING>45s → MISSED, ACCEPTED>4h → ENDED), CallSurface UI.

Disappearing modes

``OFF`` / ``TIMED`` / ``AFTER_READ``, 60..604800 сек, per-message snapshot тул policy сольсны дараа хуучин мессеж бүрэн үлдэнэ; ``/api/cron/messages-cleanup`` minute-grain delete + SSE notify.

MESSAGING НЭМЭГДСЭН FEATURE-УУД

- **Message reactions, reply, edit/delete tracking** (``MessageReaction``, ``Message.replyTold``, ``editedAt``, ``deletedAt``)
- **Group chat + roles** — ``ConversationMember.role`` (ADMIN/MEMBER), pinned messages, member block, joined time, group rename
- **Voice messages** + дуудлагын history, conversation search, pin, leave, block
- **Typing indicator + member-by-member presence**, message request inbox (allowMessageRequests)

MULTI-CHANNEL PUSH + MEDIA PIPELINE

3 push сувгийн dispatcher

Web Push (VAPID), FCM HTTP v1 (Android — service-account RS256 JWT, ``messages:send``), APNs HTTP/2 (iOS — token-based ES256 JWT). Auto-cleanup: 410 / ``BadDeviceToken`` / ``UNREGISTERED``.

Quiet hours per-TZ + Categories

``User.quietHoursStart/End/Timezone`` (IANA TZ), minute-of-day window, midnight wrap. ``alerting`` vs ``silent`` category — Web/APNs/FCM 3-тоо тус тус mapping.

Cloud storage (local/S3/R2/Blob), CDN base URL, magic-byte MIME validator, Sharp limitInputPixels, video duration check + FFmpeg transcode, EXIF strip, NSFW/virus moderation hook (ClamAV).

Per-user rate limit, alt text заавал accessibility, file-type magic detection, dimension check (DoS protection), audit log per upload.

Social граф, нээлт, performance, infrastructure

Гуравдугаар хэсэг — Social граф, контент нээлт, creator-ийн зэвсэг, performance, DevOps/observability болон GDPR нийцэл. Таницуулгад "future" гэж жагсаасан recommendation, creator analytics, mobile PWA — бүгд бодит код болсон.

SOCIAL ГРАФ БА КОНТЕНТЫН ӨРГӨТГӨЛ

Hashtag · Mention · Repost · Pin · Draft · Series
``Hashtag`` + ``PostHashtag`` (trending API + хуудас), ``Mention`` (post/story аль аль нь), Twitter-style ``Repost`` (idempotent upsert), 3-хүртэлх ``PinnedPost``, ``Draft`` бичих + scheduling, ``PostSeries`` reorder.

Story өргөтгөл
``StoryHighlight`` + items, ``StoryReaction``, ``StoryCollaborator``, stickers, ``Story.musicTrack``, ``Story.playbackMode``, ``Story.location``, expiry reminder cron.

Polls + Comment moderation + Alt
``Poll`` + ``PollVote`` (per-post poll), ``CommentModerationStatus`` enum (хүлээгдэж буй comment-уудыг pre-approve), media-д alt text заавал accessibility.

Close Circle · Favorite · Block · Mute · Report
``CloseCircle`` (private audience), ``Favorite`` (хувийн closer-than-friend), ``UserBlock`` (cascade unfollow + hide), ``Mute`` (posts/stories/messages/notifications тус тусын flag), 3-төрлийн ``Report``.

DISCOVERY + CREATOR ЗЭВСЭГ

Search + history + recommended
Users + posts + hashtags + locations advanced filters, ``SearchHistory`` DB-д, ``/api/user/recommended-follows`` нь FoF×3 + категори×2 + engagement×1.5 + log-follower heuristic ranker.

Creator Mode + XP + Leaderboard
Гэйм-механик: POST=10, LIKE=2, COMMENT=3, FOLLOW=4, DAILY=5 XP; Subscription tier multiplier; Redis leaderboard; level + badge + border unlock; verification request (blue check).

Creator Dashboard (real-data)
PostView/Like/Comment/SavedPost/Repost/Follow real aggregation, period delta %, daily views chart (raw SQL ``DATE_TRUNC``), top-5 posts, engagement breakdown, range 7d/30d/90d.

Feedback + Verification + Subscription
``Feedback`` model (category + status, 10/h limit, audit log), ``VerificationRequest`` (30-day cooldown), ``Subscription`` tier (FREE/PLUS/PRO/ELITE).

PERFORMANCE + EMAIL + CRON + I18N + GDPR

СИСТЕМ	ХЭРЭГЖИЛТ
Performance	react-window virtual scroll, code split (feed-screen → 16+ sub-component), PWA manifest + service worker, Offline mode (IndexedDB + Background Sync retry queue), bundle analyzer, hover prefetch (intent-based), Redis entity cache
Email pipeline	BullMQ queue (Redis fallback), worker script, React-email template 5 ширхэг (welcome/verify/reset/login-alert/digest), Daily/Weekly digest cadence, login-on-new-device alert

Cron jobs (7)	email-digest, publish-scheduled-posts, story-expiry-reminders, calls-cleanup (миссעד/ended), messages-cleanup (disappearing delete), audit-log-retention, hard-delete-users (GDPR grace)
i18n 6 хэл + RTL	en · mn · zh · ja · ko · ar (RTL); `User.preferredLanguage` сервер sync; email/push текстийн орчуулга (mn/en бэлэн, бусад copy-back)
GDPR компэлайнс	Cookie consent banner (Accept all / Essential only), `/legal/privacy` + `/legal/terms`, `POST /api/user/export-data`, `accountDeletionRequestedAt` + grace cron (30д default)
Observability	Sentry (server + edge + client), Pino structured logs, `web-vitals` RUM endpoint (POST `/api/metrics/web-vitals`), Health (`/api/health`) + Readiness (`/api/health/ready`), N+1 query analyzer script, CI schema drift detector

🚀 DEVOPS + INFRA (ӨМНӨ НЬ ОГТ БАЙГААГҮЙ)

- **Docker** — Multi-stage `Dockerfile` (Next standalone, non-root user), `docker-compose.yml` (Postgres + coturn), `.env.example` 200+ мөр заавартай
- **CI/CD** — GitHub Actions: Postgres service, Prisma generate + migrate deploy, shadow DB drift detection (`migrate diff --exit-code`), typecheck, build
- **Database** — Pool config (env-aap max/idle/connect timeout), PgBouncer заавар, `scripts/pg-backup.example.sh`
- **Routes** — `{username}` public profile, `{hashtag}/{tag}`, `/explore`, `/search`, `/post/{id}`, `/story/{id}`, `/series/{id}`, `/drafts`, `/saved`, `/auth/forgot-password`, `/auth/reset/{token}`, `/auth/verify-email`, `/legal/privacy`, `/legal/terms`, `/settings/billing`, `/offline`, `not-found/error/global-error/loading`
- **Refactor** — `feed-screen.tsx` → 16+ sub-component (`components/feed/`), route inconsistency `/profile?id=` → `{username}` + RESERVED_USERNAME_PATHS, landing inline style → `landing.css`, mock data 6 дэлгэцээс бүхэлд нь устгасан
- **Secrets management** — `docs/SECRETS\_ROTATION.md` runbook (JWT, refresh, CRON, Stripe, Redis товч rotate), Sentry DSN, FCM/APNs/VAPID, S3/R2/Blob тус тусын secret env

Шударга үнэлгээ

Танилцуулгаас хойш хийгдсэн ажлын **хэмжээ**-г үнэлэхэд апп нэг сарын дотор 8 model-той hobby project-оос 50+ model-той, 200+ endpoint-той, 7 cron job-той, 12-давхаргат хамгаалалттай **production-readiness 80%** түвшинд гарсан. Доорх давуу/сул тал шударгаар:

Дараах хуудсанд төслийн давуу болон сул талыг шударгаар үнэлж танилцуулъя. Эхлээд давуу талуудаас:

● Давуу талууд

• Бүтэн тех стек бодитоор хэрэгжсэн

— frontend, backend, real-time, push, calls, encryption бүгд нэг кодын сан дотор. CI/CD автомат, Docker-аар бүтэн орчин.

• Аюулгүй байдалд гүн анхаарсан

— End-to-end encryption (Web Crypto), defense-in-depth (CSP, CSRF, brute-force, password policy, refresh rotation), SQL injection эрсдэлгүй.

• Бодит цагийн боломжууд

— SSE + Redis Pub/Sub multi-instance scale-д бэлэн. WebRTC дуу/видео coturn-той production-түвшний. Multi-channel push (Web/Android/iOS).

• Гэйм-механик нэвтрүүлсэн

— Creator Mode XP систем (POST=10, LIKE=2, COMMENT=3, FOLLOW=4 + subscription multiplier) хэрэглэгчийн идэвхийг нэмэгдүүлдэг (gamification).

• Олон улсын стандарт

— i18n 6 хэлтэй (RTL дэмжлэгтэй), GDPR compliance (cookie consent, data export, privacy policy/terms).

• Ажиглалт ба чанарын баталгаа

— Sentry + pino + Web Vitals, өөрөө бичсэн N+1 query analyzer (CI integration), schema drift detection.

Гэхдээ мөн адил шударгаар сул талуудаа ч хэлэх ёстой:

● Сул талууд

• Test суурь дутуу

— Vitest/Playwright config байхгүй, `\*.test.ts` 0 ширхэг. Refactor regression-ийг баталгаатай барихгүй. *Шийдэл:* Vitest + supertest, 60%+ coverage.

• Stripe payment бүрэн биш

— Subscription model байгаа ч `stripeCustomerId/status/currentPeriodEnd` талбаргүй. Webhook scaffold (HMAC verify) бэлэн ч жинхэнэ event handler хийгдээгүй.

• E2EE хязгаарлагдмал threat model

— Per-message forward secrecy байхгүй (Double Ratchet эс хэрэгжүүлсэн). Single device per user. Safety number / QR verification UX байхгүй.

• Group call хийгдээгүй

— 1:1 only. Group call SFU (LiveKit / mediasoup) шаардана.

• SEO талаас сул

— Page-түс OpenGraph/Twitter card байхгүй, sitemap.ts, robots.txt, dynamic OG image (`/api/og/[postId]`) дутуу.

• Component refactor дутуу

— `messages-screen.tsx` (123 KB), `settings-screen.tsx` (~119 KB) гэх мэт нэг файл маш том. Maintainability буурдаг.

- **AI screen placeholder**

— провайдер integration хараахан хийгдээгүй. Audience demographics dashboard-д "Coming soon".

Тоон үзүүлэлт ба дараагийн алхам

Эцэст нь нийт хийгдсэн ажлын тоон үзүүлэлт болон цаашид хийх ажлуудыг танилцуулъя:

200+
API ENDPOINT

40+
PRISMA MODEL

6
ХЭЛ (118N)

3
PUSH CHANNEL

СҮҮЛД ХИЙГДСЭН ТОМООХОН АЖЛУУД

АЖИЛ	СТАТУС
Creator Mode + XP систем + leaderboard	DONE
E2EE migration deploy + landing.css extract	DONE
WebRTC calling + coturn + cleanup cron	DONE
Disappearing messages (TIMED · AFTER_READ)	DONE
Multi-channel push dispatcher (Web · FCM · APNs)	DONE
Mock data устрал · creator dashboard real data	DONE
Refresh token rotation · brute-force protection	DONE
Cookie consent · GDPR data export	DONE
CSRF · Security headers · CSP	DONE
Test suite (Vitest + Playwright)	TODO
Stripe payment integration	TODO
Component refactor (messages/settings/profile)	WIP

ЦААШДЫН АЖЛУУД

Хамгаалалтын дараа дараах ажлуудыг үе шаттайгаар хэрэгжүүлэхээр төлөвлөж байна:

Vitest + Playwright тест суурь
Auth, posts, messages, e2ee endpoint-уудад unit + integration тест. CI дээр coverage 60%+ заавал.

Stripe payment integration
Subscription schema ('stripeCustomerId', 'currentPeriodEnd', 'status'), webhook event handler, Settings → Billing UI, /pricing page.

Том component-уудыг refactor хийх
messages-screen → conv-list, composer, voice-recorder, bubble; settings-screen → 9 sub-page; profile-screen → header/grid/highlights/edit-modal.

SEO + Dynamic OG image
app/sitemap.ts, robots.txt, /api/og/[postId] (Next ImageResponse), per-page openGraph + twitter card, canonical URL.

LINKSY · 11 · BANKSY.AA00

Group call (SFU)

LiveKit эсвэл mediasoup-аар олон-оролцогчтой видео дуудлага; одоогийн 1:1 P2P-ийг хадгалж SFU-г сонголтоор.

libsignal-аар E2EE сайжруулах

Double Ratchet (per-message forward secrecy), multi-device, safety number verification UX.

Дүгнэлт. Linksy нь дипломын ажлын хүрээнд production-түвшний 80%-ийг гүйцээсэн. Creator mode, аюулгүй байдал, real-time, multi-platform push, end-to-end encryption бүгд бодитоор ажиллаж байна. Үлдсэн 20% — тест, payment, refactor, SEO нь хамгаалалтын дараа төлөвлөгөөт алхамаар хийгдэнэ.

Сонссон танаа баярлалаа. Асуулт байвал хариулахад бэлэн байна.